

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.preprocessing import LabelEncoder, MultiLabelBinarizer
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
In [2]: df = pd.read_csv("movie_metadata (1) (4).csv") # Load dataset/content/movie_metadata (1) (4).csv
print(df.head()) # View first few rows
```

```
   color    director_name  num_critic_for_reviews  duration \
0  Color    James Cameron                723.0      178.0
1  Color    Gore Verbinski                302.0      169.0
2  Color          Sam Mendes                602.0      148.0
3  Color  Christopher Nolan                813.0      164.0
4   NaN      Doug Walker                  NaN         NaN

   director_facebook_likes  actor_3_facebook_likes  actor_2_name \
0                   0.0                855.0  Joel David Moore
1                  563.0               1000.0   Orlando Bloom
2                   0.0                161.0    Rory Kinnear
3                22000.0              23000.0  Christian Bale
4                  131.0                  NaN    Rob Walker

   actor_1_facebook_likes  gross  genres ... \
0                1000.0  760505847.0  Action|Adventure|Fantasy|Sci-Fi ...
1               40000.0  309404152.0    Action|Adventure|Fantasy ...
2               11000.0  200074175.0    Action|Adventure|Thriller ...
3               27000.0  448130642.0    Action|Thriller ...
4                 131.0          NaN    Documentary ...

   num_user_for_reviews  language  country  content_rating  budget \
0                3054.0  English    USA      PG-13  237000000.0
1                1238.0  English    USA      PG-13  300000000.0
2                 994.0  English    UK      PG-13  245000000.0
3                2701.0  English    USA      PG-13  250000000.0
4                 NaN      NaN      NaN      NaN      NaN

   title_year  actor_2_facebook_likes  imdb_score  aspect_ratio \
0       2009.0                936.0         7.9         1.78
1       2007.0                5000.0         7.1         2.35
2       2015.0                 393.0         6.8         2.35
3       2012.0              23000.0         8.5         2.35
4         NaN                 12.0         7.1         NaN

   movie_facebook_likes
0                33000
1                   0
2                85000
3               164000
4                   0
```

[5 rows x 28 columns]

```
In [3]: print(df.isnull().sum()) # Check for missing values
df.dropna(inplace=True) # Remove null values
```

```

color                19
director_name        104
num_critic_for_reviews 50
duration             15
director_facebook_likes 104
actor_3_facebook_likes 23
actor_2_name         13
actor_1_facebook_likes 7
gross                884
genres               0
actor_1_name         7
movie_title          0
num_voted_users      0
cast_total_facebook_likes 0
actor_3_name         23
facenumber_in_poster 13
plot_keywords        153
movie_imdb_link       0
num_user_for_reviews 21
language             14
country              5
content_rating       303
budget              492
title_year           108
actor_2_facebook_likes 13
imdb_score           0
aspect_ratio         329
movie_facebook_likes 0
dtype: int64

```

```
In [4]: print(df.describe()) # Summary statistics
```

```

      num_critic_for_reviews  duration  director_facebook_likes  \
count      3755.000000    3755.000000      3755.000000
mean         167.392277    110.260186         807.463648
std          123.465521     22.649332       3068.570417
min           2.000000     37.000000         0.000000
25%           77.000000     96.000000        11.000000
50%          139.000000    106.000000        64.000000
75%          224.000000    120.000000       235.000000
max          813.000000    330.000000      23000.000000

```

```

      actor_3_facebook_likes  actor_1_facebook_likes      gross  \
count      3755.000000    3755.000000  3.755000e+03
mean         771.484953     7753.390146  5.262614e+07
std         1894.460316    15520.897163  7.032249e+07
min           0.000000         0.000000  1.620000e+02
25%          194.500000         745.000000  8.301882e+06
50%          436.000000        1000.000000  3.009311e+07
75%          691.000000       13000.000000  6.690181e+07
max         23000.000000       640000.000000  7.605058e+08

```

```

      num_voted_users  cast_total_facebook_likes  facenumber_in_poster  \
count      3.755000e+03      3755.000000      3755.000000
mean         1.058489e+05      11530.158988         1.377630
std          1.520496e+05      19123.805685         2.041689
min           9.100000e+01         0.000000         0.000000
25%          1.966300e+04      1920.500000         0.000000
50%          5.397700e+04      4060.000000         1.000000
75%          1.286110e+05      16243.000000         2.000000
max          1.689764e+06      656730.000000        43.000000

```

```

      num_user_for_reviews      budget  title_year  \
count      3755.000000  3.755000e+03  3755.000000
mean         336.914514  4.624810e+07  2002.974434
std          411.258897  2.260393e+08    9.888558
min           4.000000  2.180000e+02  1927.000000
25%          110.000000  1.000000e+07  1999.000000
50%          210.000000  2.500000e+07  2004.000000
75%          398.500000  5.000000e+07  2010.000000
max          5060.000000  1.221550e+10  2016.000000

```

```

      actor_2_facebook_likes  imdb_score  aspect_ratio  movie_facebook_likes
count      3755.000000    3755.000000    3755.000000      3755.000000
mean         2022.314248     6.464740     2.110951       9349.396272
std          4545.393731     1.055865     0.353093      21464.027749
min           0.000000     1.600000     1.180000         0.000000
25%          385.000000     5.900000     1.850000         0.000000
50%          686.000000     6.600000     2.350000        227.000000
75%          976.000000     7.200000     2.350000      11000.000000
max         137000.000000     9.300000    16.000000     349000.000000

```

```
In [5]: print("Total variables:", df.shape[1]) # Number of columns
```

Total variables: 28

```
In [6]: print(df['language'].value_counts()) # Count of languages
        print(df['director_name'].value_counts()) # Count of directors
```

```
language
English      3598
French        34
Spanish       23
Mandarin      15
German        10
Japanese      10
Cantonese     7
Italian       7
Portuguese    5
Hindi         5
Korean        5
Norwegian     4
Danish        3
Thai          3
Persian       3
Dutch         3
Indonesian    2
Dari          2
Aboriginal    2
Aramaic       1
Hungarian     1
Kazakh        1
Maya          1
Filipino      1
Mongolian     1
Czech         1
Russian       1
Zulu          1
Hebrew        1
Arabic        1
Vietnamese    1
Bosnian       1
Romanian      1
Name: count, dtype: int64
director_name
Steven Spielberg    25
Woody Allen         19
Clint Eastwood      19
Ridley Scott        17
Martin Scorsese     16
..
Noel Marshall       1
Todd Lincoln        1
Julian Jarrold      1
Peter Farrelly      1
Shane Carruth       1
Name: count, Length: 1658, dtype: int64
```

```
In [7]: hindi_movies = df[df['language'] == 'Hindi']
        print(hindi_movies.head()) # Display Hindi movies
        hindi_movies.to_csv("hindi_movies.csv", index=False) # Save for further analysis
```

	color	director_name	num_critic_for_reviews	duration	\
3075	Color	Karan Johar	20.0	193.0	
3276	Color	Yash Chopra	50.0	176.0	
3344	Color	Karan Johar	210.0	128.0	
4385	Color	Ritesh Batra	195.0	104.0	
4490	Color	Mira Nair	137.0	114.0	

	director_facebook_likes	actor_3_facebook_likes	actor_2_name	\
3075	160.0	860.0	John Abraham	
3276	147.0	1000.0	Katrina Kaif	
3344	160.0	81.0	Jimmy Shergill	
4385	25.0	73.0	Nimrat Kaur	
4490	300.0	73.0	Randeep Hooda	

	actor_1_facebook_likes	gross	genres	...	\
3075	8000.0	3275443.0	Drama	...	
3276	8000.0	3047539.0	Drama Romance	...	
3344	8000.0	4018695.0	Adventure Drama Thriller	...	
4385	638.0	4231500.0	Drama Romance	...	
4490	307.0	13876974.0	Comedy Drama Romance	...	

	num_user_for_reviews	language	country	content_rating	budget	\
3075	264.0	Hindi	India	R	700000000.0	
3276	286.0	Hindi	India	Not Rated	7217600.0	
3344	235.0	Hindi	India	PG-13	12000000.0	
4385	162.0	Hindi	India	PG	1000000.0	
4490	214.0	Hindi	India	R	7000000.0	

	title_year	actor_2_facebook_likes	imdb_score	aspect_ratio	\
3075	2006.0	1000.0	6.0	2.35	
3276	2012.0	3000.0	6.9	2.35	
3344	2010.0	327.0	8.0	2.35	
4385	2013.0	85.0	7.8	2.35	
4490	2001.0	209.0	7.4	1.85	

	movie_facebook_likes
3075	659
3276	12000
3344	27000
4385	16000
4490	0

[5 rows x 28 columns]

```
In [8]: top_movies = df[df['imdb_score'] >= 8]
print(top_movies.head()) # Display top-rated movies
```

	color	director_name	num_critic_for_reviews	duration	\
3	Color	Christopher Nolan	813.0	164.0	
17	Color	Joss Whedon	703.0	173.0	
27	Color	Anthony Russo	516.0	147.0	
43	Color	Lee Unkrich	453.0	103.0	
47	Color	Bryan Singer	539.0	149.0	

	director_facebook_likes	actor_3_facebook_likes	actor_2_name	\
3	22000.0	23000.0	Christian Bale	
17	0.0	19000.0	Robert Downey Jr.	
27	94.0	11000.0	Scarlett Johansson	
43	125.0	721.0	John Ratzenberger	
47	0.0	20000.0	Peter Dinklage	

	actor_1_facebook_likes	gross	\
3	27000.0	448130642.0	
17	26000.0	623279547.0	
27	21000.0	407197282.0	
43	15000.0	414984497.0	
47	34000.0	233914986.0	

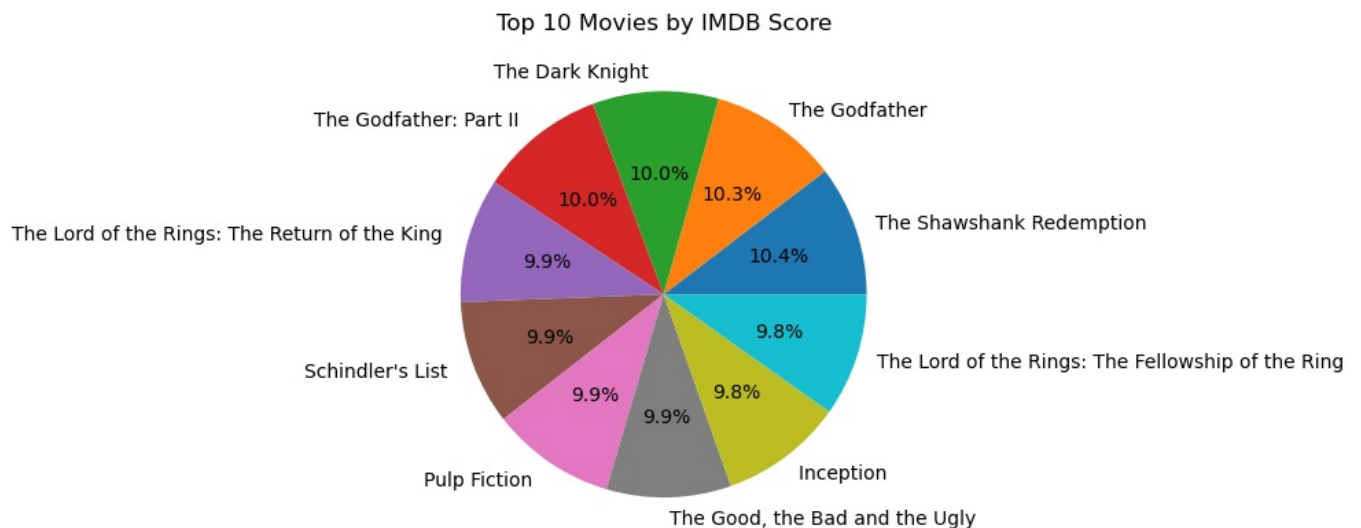
	genres	... num_user_for_reviews	\
3	Action Thriller	...	2701.0
17	Action Adventure Sci-Fi	...	1722.0
27	Action Adventure Sci-Fi	...	1022.0
43	Adventure Animation Comedy Family Fantasy	...	733.0
47	Action Adventure Fantasy Sci-Fi Thriller	...	752.0

	language	country	content_rating	budget	title_year	\
3	English	USA	PG-13	250000000.0	2012.0	
17	English	USA	PG-13	220000000.0	2012.0	
27	English	USA	PG-13	250000000.0	2016.0	
43	English	USA	G	200000000.0	2010.0	
47	English	USA	PG-13	200000000.0	2014.0	

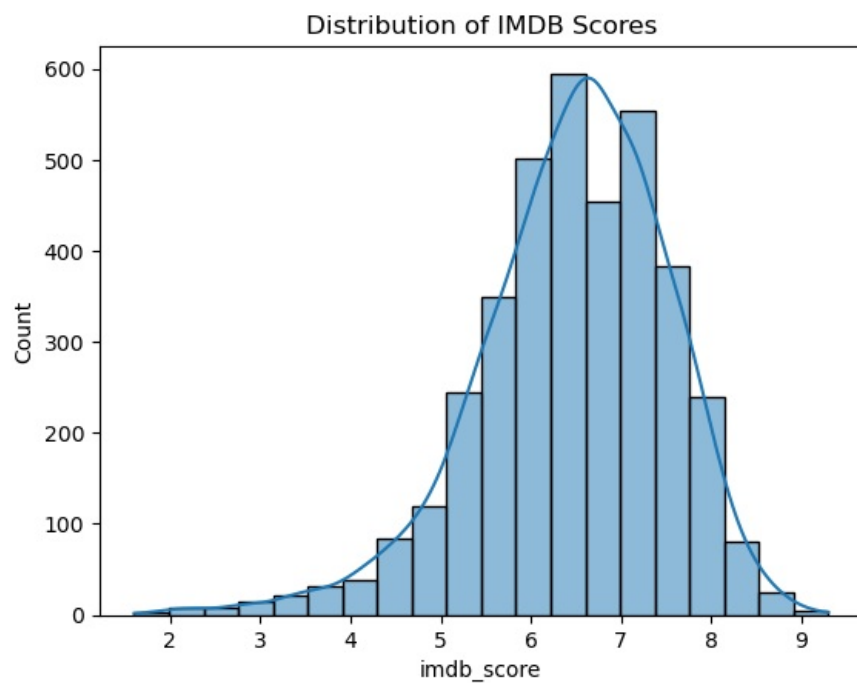
	actor_2_facebook_likes	imdb_score	aspect_ratio	movie_facebook_likes
3	23000.0	8.5	2.35	164000
17	21000.0	8.1	1.85	123000
27	19000.0	8.2	2.35	72000
43	1000.0	8.3	1.85	30000
47	22000.0	8.0	2.35	82000

[5 rows x 28 columns]

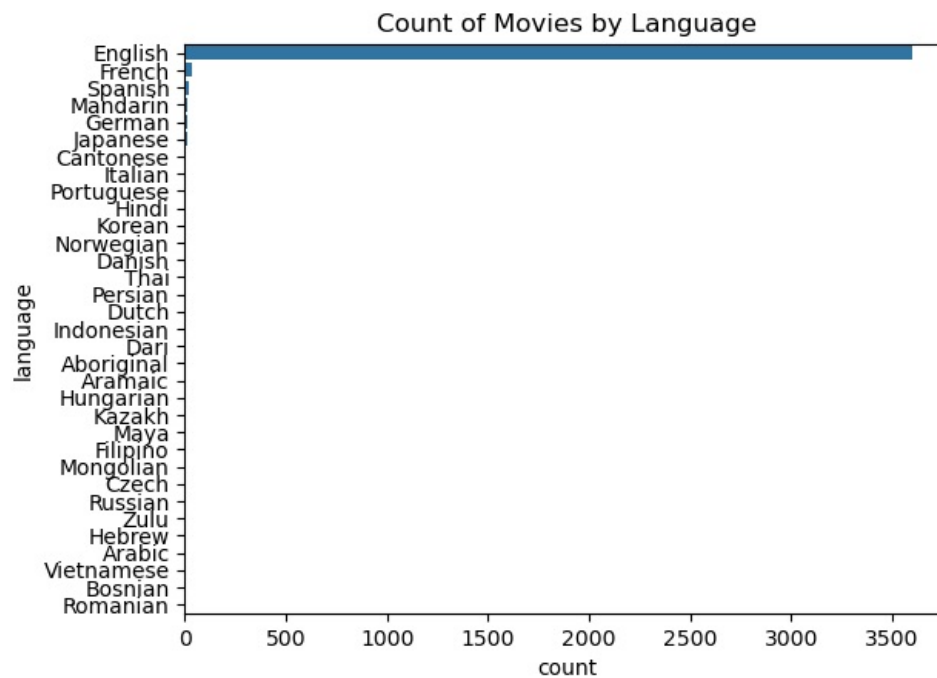
```
In [9]: top_10 = df.nlargest(10, 'imdb_score')
plt.pie(top_10['imdb_score'], labels=top_10['movie_title'], autopct='%1.1f%%')
plt.title("Top 10 Movies by IMDB Score")
plt.show()
```



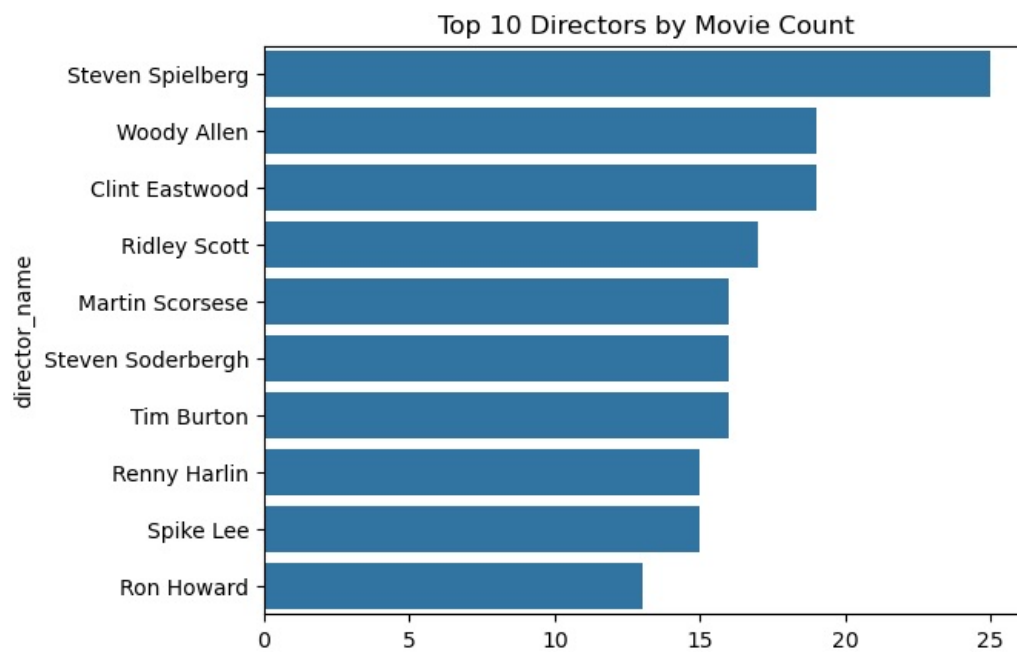
```
In [10]: sns.histplot(df['imdb_score'], bins=20, kde=True)
plt.title("Distribution of IMDB Scores")
plt.show()
```



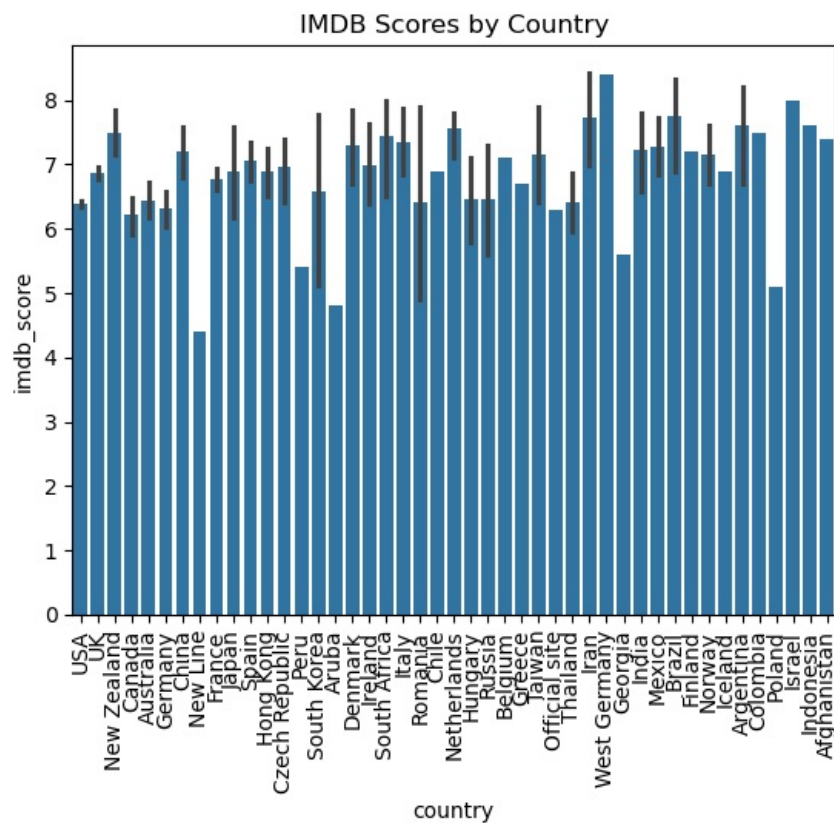
```
In [11]: sns.countplot(y=df['language'], order=df['language'].value_counts().index)
plt.title("Count of Movies by Language")
plt.show()
```



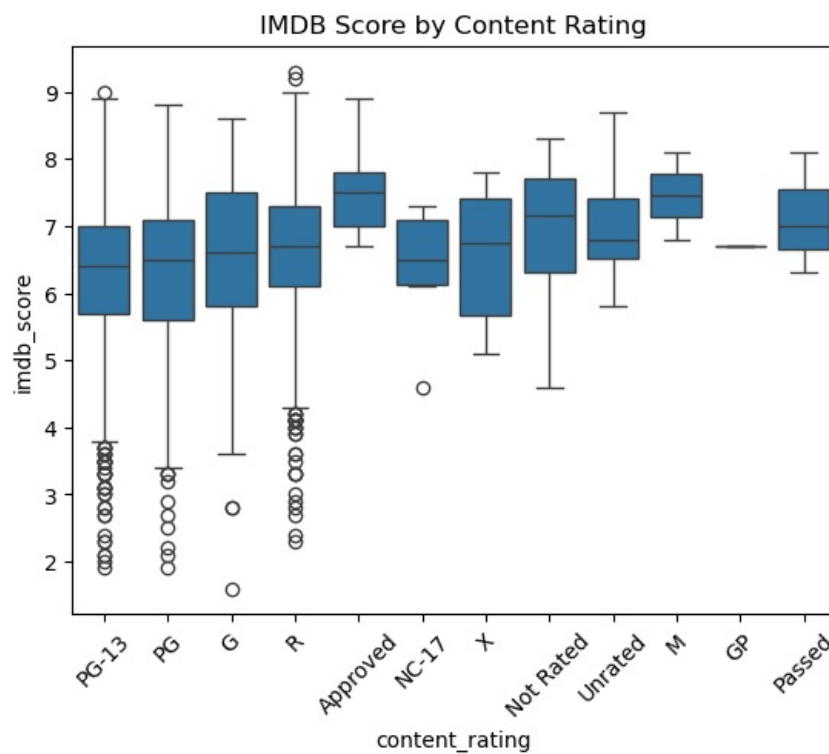
```
In [12]: top_directors = df['director_name'].value_counts().nlargest(10)
sns.barplot(y=top_directors.index, x=top_directors.values)
plt.title("Top 10 Directors by Movie Count")
plt.show()
```



```
In [13]: sns.barplot(x=df['country'], y=df['imdb_score'])
plt.xticks(rotation=90)
plt.title("IMDB Scores by Country")
plt.show()
```



```
In [14]: sns.boxplot(x=df['content_rating'], y=df['imdb_score'])
plt.xticks(rotation=45)
plt.title("IMDB Score by Content Rating")
plt.show()
```



```
In [15]: #Check for null values
df.fillna(df.median(numeric_only=True), inplace=True) # Fill numeric NaNs with median
df.dropna(subset=['director_name', 'language', 'country'], inplace=True) # Drop rows with missing categorical

In [16]: label_encoder = LabelEncoder()
categorical_cols = ['color', 'director_name', 'actor_2_name', 'actor_1_name', 'actor_3_name', 'language', 'country']
for col in categorical_cols:
    df[col] = label_encoder.fit_transform(df[col].astype(str))

In [17]: mlb_genres = MultiLabelBinarizer()
genres_encoded = mlb_genres.fit_transform(df['genres'].astype(str).str.split('|'))
genres_df = pd.DataFrame(genres_encoded, columns=mlb_genres.classes_)
df = pd.concat([df, genres_df], axis=1)
df.drop(['genres'], axis=1, inplace=True)

mlb_keywords = MultiLabelBinarizer()
keywords_encoded = mlb_keywords.fit_transform(df['plot_keywords'].astype(str).str.split('|'))
keywords_df = pd.DataFrame(keywords_encoded, columns=mlb_keywords.classes_)
df = pd.concat([df, keywords_df], axis=1)
df.drop(['plot_keywords'], axis=1, inplace=True)

In [18]: df.drop(['movie_imdb_link', 'aspect_ratio', 'movie_title'], axis=1, inplace=True)

In [19]: X = df.drop('imdb_score', axis=1)
y = df['imdb_score']

In [20]: X.fillna(0, inplace=True)

In [21]: y.fillna(0, inplace=True)

In [22]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

In [23]: models = {
    'Linear Regression': LinearRegression(),
    'Decision Tree': DecisionTreeRegressor(),
    'Random Forest': RandomForestRegressor()
}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(f"{name} Performance:")
    print("MSE:", mean_squared_error(y_test, y_pred))
    print("MAE:", mean_absolute_error(y_test, y_pred))
    print("R2 Score:", r2_score(y_test, y_pred))
    print("-"*40)
```


Linear Regression Performance:

MSE: 1.2294232832178662

MAE: 0.8150588634255107

R2 Score: 0.8192266299255695

Decision Tree Performance:

MSE: 0.7800437158469945

MAE: 0.5774863387978142

R2 Score: 0.8853030252119813

Random Forest Performance:

MSE: 0.38403586666666656

MAE: 0.41353005464480874

R2 Score: 0.9435316877478679

In []: